עזריאלי
מכללה אקדמית להנדסה
ירושלים

Insecure Deserialization Attack in Web Applications

מוחמד דוויק

הנדסת תוכנה

קורס אבטחת מערכות WEB

25/2/2026

תיאור החולשה

חולשת **Insecure Deserialization** מתרחשת כאשר מערכת מבצעת דה-סריאליזציה לנתונים שמקורם במשתמש ללא אימות תקינות או שלמות המידע. סריאליזציה היא תהליך הממיר מבני נתונים מורכבים לפורמט הניתן להעברה או אחסון, ואילו דה-סריאליזציה מחזירה את הנתונים למבנה המקורי בזיכרון התוכנית.

כאשר השרת סומך על נתונים שמגיעים מצד הלקוח ומשתמש בהם לצורך קבלת החלטות הרשאה, תוקף יכול לשנות את הנתונים ולגרום למערכת לפעול בהתאם למידע מזויף. חולשה זו עלולה להוביל לעליית הרשאות, גישה למידע רגיש ואף להרצת קוד זדוני.

סביבת העבודה והטכנולוגיות

המערכת פותחה כהדגמת Proof of Concept באמצעות:

- Python ו-Flask בצד השרת.
- React בצד הלקוח.
- שימוש ב-Cookies לניהול Session.
- Pickle – מודול מובנה ב-Python לביצוע סריאליזציה ודה-סריאליזציה של אובייקטים, הממיר אותם לייצוג בינאר לצורך אחסון או העברה. שימוש בו על נתונים לא מהימנים עלול להיות מסוכן.
- שימוש ב-Browser Developer Tools לצורך צפייה ועריכה של ערכי ה-Cookie בדפדפן.

תיאור ההדגמה

במהלך ההתחברות למערכת, השרת יוצר אובייקט Session המכיל שם משתמש ורמת הרשאות. אובייקט זה עובר סריאליזציה באמצעות Pickle ונשמר בתוך Cookie בדפדפן המשתמש.

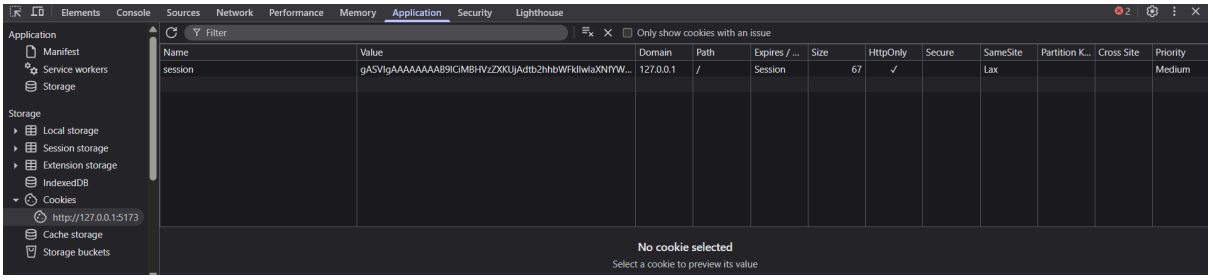
בכל בקשה למערכת, השרת מבצע דה-סריאליזציה ל-Cookie ומשתמש במידע שבתוכו לצורך אימות והרשאה.

שלבי המתקפה:

1. המשתמש מתחבר למערכת והשרת יוצר אובייקט Session המכיל שם משתמש והרשאות, ולאחר מכן מבצע סריאליזציה ושומר אותו בתוך Cookie בצד הלקוח.

2. השרת סומך על תוכן ה-Cookie כמקור אמת ואינו מבצע אימות שלמות (Integrity Check) לפני דה-סריאליזציה.

3. התוקף מעתיק את ערך ה-Cookie באמצעות כלי הפיתוח בדפדפן.



4. התוקף מפענח את תוכן ה-Cookie, מזהה את מבנה האובייקט ושדות ההרשאה (כגון is_admin).

```
PS C:\Users\moham\OneDrive\Desktop\jce\web_security\final-project-mohamad-dweik\insecure_desr_poc> python forge_cookie.py recon --cookie gASVIgAAAAAAAAB9lCiMBHVzZXKUjA
dtb2hhbWFklIwIaXNfYWRtaW6UiXUu

Decoded session object:
{'user': 'mohamad', 'is_admin': False}

Discovered fields:
user = mohamad
is_admin = False
```

לדוגמה, לאחר פענוח ה-Cookie התוקף רואה אובייקט Session במבנה הבא (לפני שינוי):

- user: mohamad
- is_admin: False

5. התוקף משנה את ערך ההרשאה ל-True, מבצע סריאליזציה מחדש, ומחליף את ערך ה-Cookie בדפדפן.

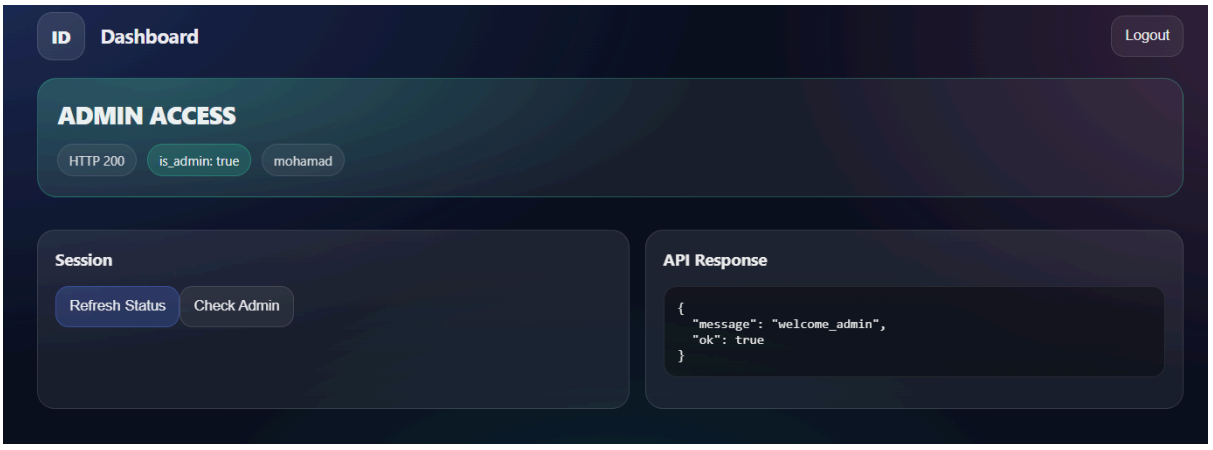
```
PS C:\Users\moham\OneDrive\Desktop\jce\web_security\final-project-mohamad-dweik\insecure_desr_poc> python forge_cookie.py forge --user mohamad --admin
>>
gASVIgAAAAAAAAB9lCiMBHVzZXKUjAdtb2hhbWFklIwIaXNfYWRtaW6UiHUu
```

לאחר שינוי:

- user: mohamad
- is_admin: True

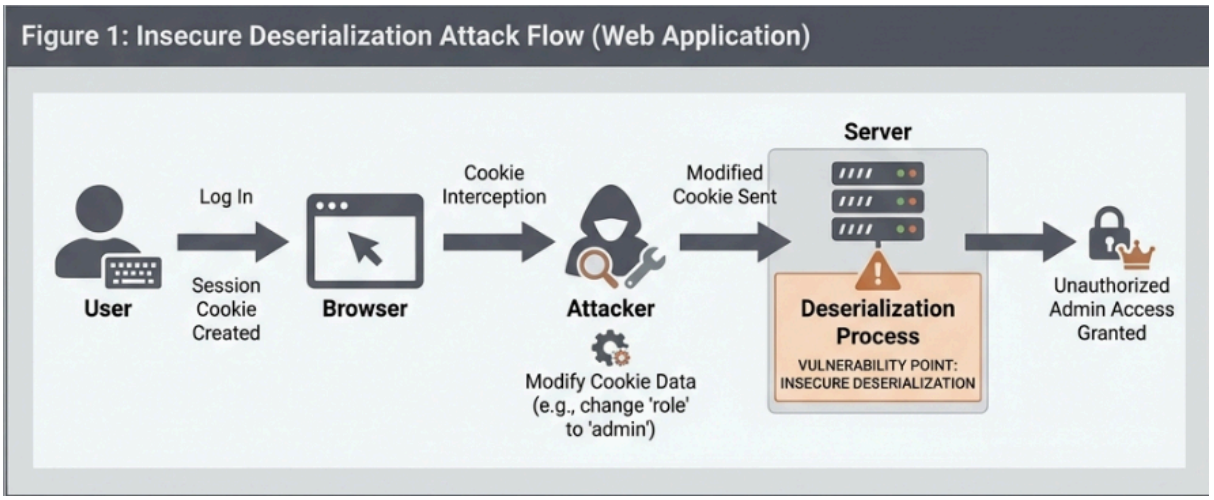
6. התוקף מחליף את ה-Cookie בדפדפן ושולח בקשה לשרת.

7. בעת שליחת בקשה לשרת למשל (/api/admin), השרת מבצע דה-סריאליזציה לנתונים המזויפים ללא אימות שלמות, מזהה את המשתמש כמנהל ומעניק גישה לא מורשית.



תוצאת המתקפה:

המתקפה מאפשרת לתוקף לעקוף מנגנוני הרשאה ולשלוט בזהות המשתמש. במקרה שנבדק בפרויקט, שינוי ערך הרשאות בתוך ה-Cookie אפשר גישה לאזורי Admin ללא אימות נוסף מצד השרת.



התרשים מדגים את זרימת המתקפה החל מיצירת ה-Session ועד קבלת גישה לא מורשית.

מנגנוני הגנה

החולשה נגרמה כתוצאה מאמון בנתונים הנשלטים על ידי המשתמש ללא מנגנון אימות שלמות (Integrity). אובייקט ה-Session נשמר בתוך Cookie בצד הלקוח ללא חתימה קריפטוגרפית וללא בדיקת תקינות לפני הדה-סריאליזציה. כתוצאה מכך, השרת לא היה מסוגל לזהות שינוי זדוני בנתונים ואפשר לתוקף לעקוף את מנגנון ההרשאה.

ניתן למנוע חולשה זו באמצעות:

- שימוש בפורמט JSON במקום Pickle
- הוספת חתימה קריפטוגרפית לנתוני Session
- אחסון נתוני Session בצד השרת
- ביצוע ולידציה לנתונים לפני דה-סריאליזציה

יישום שיטות אלו מונע מהתוקף לשנות את נתוני ה-Session ומקטין משמעותית את הסיכון למתקפה.

Best Practices

- לא לבצע דה-סריאליזציה לנתונים לא מהימנים באמצעות Pickle.
- להגן על נתוני Session בצד הלקוח באמצעות חתימה קריפטוגרפית (כגון HMAC).
- להעדיף אחסון Session בצד השרת (Server-side session).
- לבצע ולידציה מחמירה לנתונים לפני עיבוד.

סיכום

פרויקט זה מדגים כיצד החלטה תכנונית שגויה (אחסון נתוני Session רגישים בצד הלקוח ללא מנגנון אימות שלמות) עלולה להוביל לפגיעה חמורה באבטחת יישומי Web. באמצעות שינוי נתוני ההרשאות בתוך ה-Cookie, תוקף יכול לבצע העלאת הרשאות ולעקוף מנגנוני אבטחה ללא צורך בפריצה ישירה לשרת.

ההדגמה ממחישה את החשיבות שבאי אמון בנתונים המגיעים מהלקוח ואת הצורך ביישום מנגנוני הגנה כגון חתימה קריפטוגרפית ואחסון נתונים רגישים בצד השרת. תכנון מאובטח כבר בשלבי הפיתוח הראשוניים הוא גורם קריטי במניעת חולשות מסוג זה במערכות אמיתיות.

מקורות

OWASP – Insecure Deserialization
https://owasp.org/www-community/vulnerabilities/Insecure_Deserialization

PortSwigger Web Security Academy – Insecure deserialization
https://portswigger.net/web-security/deserialization

Invicti – Insecure Deserialization in Web Applications
https://www.invicti.com/blog/web-security/insecure-deserialization-in-web-applications/

Packetlabs – Understanding Deserialization Attacks
https://www.packetlabs.net/posts/understanding-deserialization-attacks/